

Real Dataset ML Protection 132 kV Transmission Feeder

MATLAB/Simulink Time-Domain Data → Phasor Features → Random Forest Classification & Regression

45 Simulink .mat files (v7.3 HDF5)

45 fault cases + 100 healthy cases = 145 rows

Tasks: Fault Classification | Zone Detection | IDMT Trip Time

CV Accuracy: 100% / 100% | MAE: 6–40 ms

DEV'S

Power System Protection using ML — Project Summary

Overview

ML-based protection for a **132 kV radial transmission feeder** (two-section: 50 km + 40 km) with IDMT overcurrent relays at Bus A (R1) and Bus B (R2). All models trained on **physics-simulated dataset** (4800 cases).

System Model (src/power_system.py)

- **Voltage:** 132 kV LL, 50 Hz
- **Source:** 3500 MVA, X/R = 10 (Thevenin equivalent)
- **Lines:** ACSR Panther class, $z_{\text{L}} = 0.12 + j0.40 \, \Omega/\text{km}$, $z_{\text{M}} = 0.35 + j1.25 \, \Omega/\text{km}$
- **Load:** 40 MVA, 0.85 pf lagging at Bus C (constant-Z model)
- **CTs:** R1 = 400/1, R2 = 300/1
- **Fault analysis:** Symmetrical components (Fortescue) — 10 shunt fault types + healthy

Fault Category	Types
L-G (80% real)	AG, BG, CG
L-L	AB, BC, CA
L-L-G	ABG, BCG, CAG
L-L-L	ABC

IDMT Relay & Coordination (src/idmt_relay.py)

- **Curve:** IEC 60255 Standard Inverse (SI) — $k=0.14$, $\alpha=0.02$
- **Pickup:** $1.25 \times \text{max load current}$ (R1 slightly higher for selectivity)
- **Coordination:** Time grading, CTI = 0.3 s
- R2 (downstream): TMS = 0.05 (fastest)
- R1 (upstream): TMS auto-calculated from grading point (max 3 ϕ fault at Bus B)

Simulated Dataset (src/generate_dataset.py)

Parameter	Values	Count
Fault types	10	10
Zones	1, 2	2
Locations/zone	2% ... 98% (12 steps)	12
Fault resistance (Rf)	0.01, 1, 5, 10, 25, 50 Ω	6
Loading	0.6, 1.0, 1.2 pu	3
Fault cases		4,320
Healthy cases	varied loading (0.3–1.2 pu)	480
Total rows		4,800

Noise: 1% multiplicative Gaussian on magnitudes, 0.5° on angles (CT/CVT + DFT error)

Features (28) — Only relay-measurable quantities

R1_Ia, R1_Ib, R1_Ic, R1_I0, R1_I1, R1_I2,
R1_Ia_ang, R1_Ib_ang, R1_Ic_ang,
R2_Ia, R2_Ib, R2_Ic, R2_I0, R2_I1, R2_I2,
R2_Ia_ang, R2_Ib_ang, R2_Ic_ang,
VA_a, VA_b, VA_c, VA_V0, VA_V2,
VB_a, VB_b, VB_c, VB_V0, VB_V2

Labels (7)

Label	Type	Description
fault	binary	0 = healthy, 1 = fault
fault_type	11-class	NONE + 10 fault types
zone	3-class	0=none, 1=Line1, 2=Line2
correct_relay	3-class	NONE, R1, R2 (which relay should operate)
distance_km	regression	Distance from Bus A (0–90 km)
t_R1, t_R2	regression	IDMT operating times (s)
coordinated	binary	Coordination held?

ML Tasks & Results (src/train_models.py)

Task	Type	Best Model	Test Result
M1 Fault Detection	Binary classification	DecisionTree	100% acc
M2 Fault Classification	11-class	RandomForest	100% acc, CV 100%
M3 Relay Selection	3-class	MLP	100% test / 99.6% CV
M4 Fault Location	Regression (km)	MLP	MAE 0.75 km, R ² 0.998
M5 IDMT Trip Time	Regression (s)	RandomForest	MAE 1.9 ms, R ² 0.999

Training protocol: Stratified 80/20 split + 5-fold CV **Models compared:** DecisionTree, RandomForest, SVM-RBF, MLP (classification); RF + MLP (regression) **Saved artifacts:** results/models.joblib, results/test_splits.joblib, results/metrics.json

Outputs

- **Figures** (7, 300 dpi): results/figures/fig1-fig7.png
- **Demo:** python src/demo.py (random or user-specified fault → full ML pipeline response)
- **Report:** PowerSystemProtection_ML_Report.pdf (built from docs/ via build_pdf.py)

Key Assumptions / Limitations

1. **Single-source radial system** — no parallel infeeds, no mesh
2. **Steady-state phasors only** — no transients, no harmonics, no CT saturation
3. **Perfect phasor estimation** — only 1%/0.5° noise added post-simulation
4. **Constant-Z load** — no motor contribution, no dynamic load models
5. **Bolted fault at busbar for coordination** — grading point = max 3φ at Bus B
6. **R2 sees zero fault current for Zone-1 faults** — simplified (load only at 30%)

Ready for Real Data

The pipeline is **modular** — to switch to real data:

1. Replace `data/fault_dataset.csv` with your dataset (same 28 features + 7 labels)
2. Retrain: `python src/train_models.py`
3. Regenerate plots: `python src/make_plots.py`
4. Test demo: `python src/demo.py`

Next steps: Share your real dataset format and requirements for the new model.

DEV'S

Real Dataset ML Protection — Summary

Overview

Converted **45 MATLAB/Simulink .mat files** (v7.3 format) from time-domain simulation to a **145-row CSV dataset** and trained **Random Forest models** for three protection tasks.

Data Source

Location: `Copy of Fault_Dataset_All/Fault_Dataset_All/`

Dimension	Values	Count
Zones	Z1, Z2, Z3	3
Positions	25%, 50%, 75%	3
Fault Types	LG, LL, LLG, LLL, LLLG	5
Total .mat files		45

Each .mat file contains:

- **Time-domain signals:** 0–0.2s at 500 kHz (100,001 samples)
- **5 current measurement points:** R1, R2, R3, Sending End, Receiving End
- **3 phases each** (A, B, C)
- **Metadata:** faultType, position, zone

Conversion Pipeline (`src/convert_matlab_dataset.py`)

Signal Processing

1. **Extract steady-state fault window:** 0.12–0.14s (post-fault inception)
2. **Compute phasors** via DFT at 50 Hz fundamental
3. **Calculate sequence components** (I0, I1, I2) from phase phasors
4. **Map to training features:** R1, R2 currents (magnitudes + angles) + sequence components

Output Features (28) — Match training script exactly

```
R1_Ia, R1_Ib, R1_Ic, R1_I0, R1_I1, R1_I2,  
R1_Ia_ang, R1_Ib_ang, R1_Ic_ang,  
R2_Ia, R2_Ib, R2_Ic, R2_I0, R2_I1, R2_I2,  
R2_Ia_ang, R2_Ib_ang, R2_Ic_ang,  
VA_a, VA_b, VA_c, VA_V0, VA_V2,  
VB_a, VB_b, VB_c, VB_V0, VB_V2
```

Labels (7) — From physics model + relay coordination

- `fault_type`: AG, BC, BCG, ABC, NONE
- `zone`: 0, 1, 2, 3
- `correct_relay`: NONE, R1, R2
- `t_R1`, `t_R2`: IDMT trip times (seconds)
- `distance_km`, `Rf`, `load_factor`, `coordinated`

Dataset Statistics

Split	Rows	Fault Types	Zones
Fault cases	45	AG(9), BC(9), BCG(9), ABC(18)	Z1(15), Z2(15), Z3(15)
Healthy	100	NONE	0
Total	145	5 classes	4 classes

Output: `data/fault_dataset_real.csv`

Trained Models (`src/train_real_models.py`)

Task 1: Fault Classification (5 classes)

- **Model:** RandomForest (500 trees, balanced class weights)
- **CV Accuracy:** 100% (5-fold, stratified)
- **Confusion Matrix:** Perfect diagonal

Task 2: Zone Detection (4 classes)

- **Model:** RandomForest (500 trees, balanced class weights)
- **CV Accuracy:** 100% (5-fold, stratified)
- **Confusion Matrix:** Perfect diagonal

Task 3: Operating Time Regression

Target	Cases	CV MAE	CV R ²
R1 Trip Time	15 (Z1 faults)	40.1 ms	0.897
R2 Trip Time	30 (Z2, Z3 faults)	6.3 ms	0.867

Note: Small dataset → used cross-validation (no hold-out test set)

Saved Artifacts

- `results_real/models_real.joblib` — 4 trained pipelines
- `results_real/encoders_real.joblib` — LabelEncoder for fault types
- `results_real/metrics_real.json` — CV metrics

Demo Results (`src/demo_real.py`)

Zone 1 Fault (AG, Z1, 75%)

Metric	Physics	ML	Error
Fault Type	AG	AG	✓
Zone	1	1	✓
R1 Time	0.467s	0.465s	1.7 ms
R2 Time	N/A	0.222s	—
Relay	R1	R2*	✗

Zone 1: R2 should not operate. ML predicts R2 time since regression trained on all fault cases.

Zone 2 Fault (BC, Z2, 50%)

Metric	Physics	ML	Error
Fault Type	BC	BC	✓
Zone	2	2	✓
R1 Time	0.629s	0.629s	0.6 ms
R2 Time	0.181s	0.183s	2.0 ms
Relay	R2	R2	✓
Coord. Margin	0.449s	0.446s	2.6 ms

Zone 3 Fault (AG, Z3, 25%)

Metric	Physics	ML	Error
Fault Type	AG	AG	✓
Zone	3	3	✓
R1 Time	0.896s	0.886s	9.9 ms
R2 Time	0.250s	0.247s	3.0 ms
Relay	R2	R2	✓
Coord. Margin	0.646s	0.640s	6.4 ms

Key Findings

1. **Classification tasks perfect** — Features from Simulink time-domain data are highly separable
2. **Trip time regression good** — $R^2 \sim 0.87\text{--}0.90$, MAE $\sim 6\text{--}40$ ms
3. **Relay selection works for Z2/Z3** — Correctly identifies R2 as primary
4. **Zone 1 R2 time prediction** — Model predicts time even when R2 shouldn't operate (regression trained on all cases)

Usage

```
# Convert MATLAB -> CSV
python src/convert_matlab_dataset.py

# Train models
python src/train_real_models.py

# Demo (random case)
python src/demo_real.py

# Demo (specific: fault_type zone position)
python src/demo_real.py AG 1 0.25
python src/demo_real.py BC 2 0.5
python src/demo_real.py BCG 3 0.75
```

Next Steps

1. **Augment with simulated data** — Combine 145 real + 4800 simulated for more robust regression
2. **Add fault resistance variation** — Current data only has $R_f=0.01\Omega$ (bolted)
3. **Add loading variation** — Current data only at nominal loading
4. **Directional relay logic** — Use sequence components (I_2 , I_0) for directional element
5. **COMTRADE export** — Convert for standard relay testing tools